

Chapter 4

Transform Coding



Abstract The essence of transform coding is to transform signals from one domain (e.g., time domain) to another domain (e.g., frequency domain) with a set of orthogonal basis. The transform is beneficial to eliminate signal correlation and reduce data redundancy. In this chapter, we will introduce some important transforms, including the commonly used discrete cosine transform (DCT), the wavelet transform, and the graph Fourier transform (GFT). We also show several applications of transform-based methods in point cloud attribute compression.

Keywords Transform coding · Discrete cosine transform · Wavelet transform · Lifting transform · Graph Fourier transform

4.1 Introduction

Most signals, such as images, videos, point clouds, and so on, contain relatively flat areas where the content changes slowly. After adopting appropriate transform technology for these signals, they are converted from the original time domain to the frequency domain, and the signal distribution also becomes relatively concentrated in the new representation domain. That is, the flat region of the signal is concentrated in the low-frequency region and accounts for most of the energy. In contrast, the detail of the signal is mapped to the high-frequency region and accounts for a small amount of energy. In order to reduce redundancy, transform technology is further combined with quantization, entropy coding, and other coding technologies, which can achieve efficient compression. Therefore, as an important coding technology, transform is widely used in image, video, speech, and other signal compression algorithms.

Transform techniques include nonlinear and linear transform. The former is usually implemented with neural networks [1]. As for the latter, we discuss common orthogonal transform. It can transform statistically closely related pixel value matrices into statistically independent transform coefficient matrices. After the orthogonal transform, the distribution of data will change. The transform coefficient distribution in the transform domain is usually concentrated on the low-frequency

region. The transform reduces the amount of data and provides conditions for further compression. Moreover, the orthogonal transform does not change the entropy of the information source, i.e., there is no loss of signal information before and after the transform. Thus, the original information can be obtained through inverse transform.

The ability of de-correlation and the algorithm complexity are always considered to evaluate the performance of the transform coding algorithm. Karhunen-Loève transform (KLT) is the optimal transform for de-correlating, but it is not adopted in the coding algorithm due to its high complexity. DCT is a simple transform operation, and its performance is second only to KLT. Therefore, DCT is widely used in encoders. DCT is suitable for regular image structure, but it is not skilled in transforming irregular signals, e.g., 3D point clouds. Moreover, the essence of DCT and its variants is GFT. Therefore, we will introduce the application of GFT in point cloud compression. However, the realization complexity of GFT is relatively high. Therefore, we also introduce the wavelet transform that is a low-complexity transform algorithm commonly used in point cloud compression.

4.2 Discrete Cosine Transform

4.2.1 Introduction of DCT

The discrete cosine transform (DCT) consists of the sum of cosine functions of different frequencies [2]. In many coding algorithms, DCT is adopted because of its nearly optimal asymptotic theoretical coding gain [3]. Assume we have a 1-D sequence, $f(n)$, $n = 0, 1, \dots, N - 1$, the most common DCT [4] can be defined as:

$$F(u) = a(u) \sum_{n=0}^{N-1} f(n) \cos \frac{(2n+1)u\pi}{2N}. \quad (4.1)$$

The transform coefficient is $F(u)$ for $u = 0, 1, \dots, N - 1$. The inverse discrete cosine transform (IDCT) is defined as:

$$f(n) = \sum_{u=0}^{N-1} a(u) F(u) \cos \frac{(2n+1)u\pi}{2N}. \quad (4.2)$$

In Eqs. (4.1) and (4.2), $a(u)$ is defined as:

$$a(u) = \begin{cases} \sqrt{\frac{1}{N}}, & u = 0, \\ \sqrt{\frac{2}{N}}, & \text{otherwise.} \end{cases} \quad (4.3)$$

For $u = 0$, the first transform coefficient, $F(0) = \frac{1}{N} \sum_{n=0}^{N-1} f(n)$, is the average value of the input sequence, which is called the DC Coefficient. All other transform coefficients are called AC Coefficients.

DCT can also be assembled in matrix form as:

$$\begin{aligned}
 & \begin{bmatrix} F(0) \\ F(1) \\ \vdots \\ F(N-1) \end{bmatrix} \\
 &= \frac{1}{\sqrt{N}} \begin{bmatrix} 1 & 1 & \dots & 1 \\ \sqrt{2} \cos \frac{\pi}{2N} & \sqrt{2} \cos \frac{3\pi}{2N} & \dots & \sqrt{2} \cos \frac{(2N-1)\pi}{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \sqrt{2} \cos \frac{(N-1)\pi}{2N} & \sqrt{2} \cos \frac{(N-1)3\pi}{2N} & \dots & \sqrt{2} \cos \frac{(N-1)(2N-1)\pi}{2N} \end{bmatrix} \\
 & \begin{bmatrix} f(0) \\ f(1) \\ \vdots \\ f(N-1) \end{bmatrix}. \tag{4.4}
 \end{aligned}$$

The basis set for the discrete cosine transform is shown in Fig. 4.1. We can observe that the variation increases in a top-down manner.

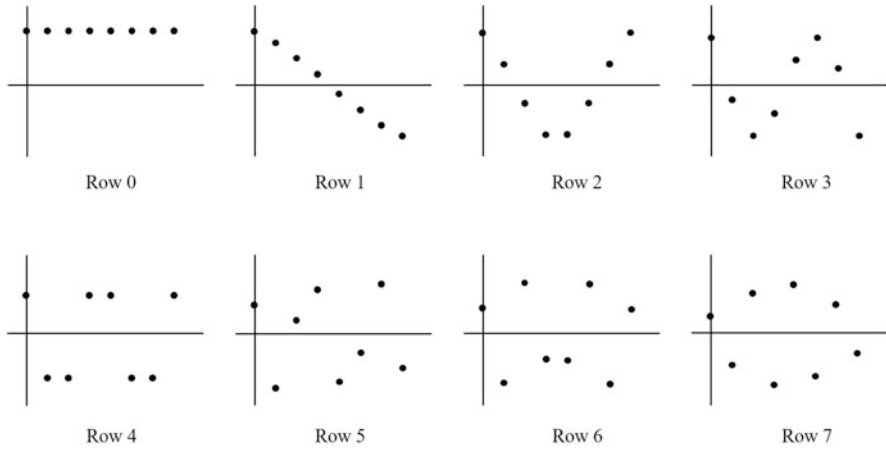


Fig. 4.1 The basis set for DCT when $N = 8$. (Source: Author)

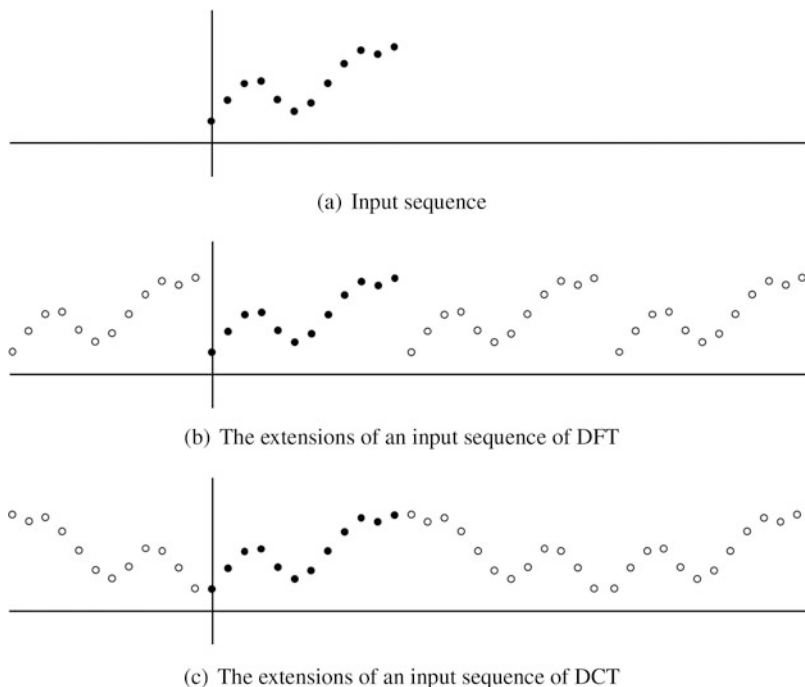


Fig. 4.2 The comparison of extensions of the input sequence between DCT and DFT. (Source: Author)

DCT is a Fourier-related transform similar to the discrete Fourier transform (DFT), while the former has better compression performance. One reason is that compared with DFT, DCT generally has the assumption of a periodical and symmetrical extension of input sequences in order to avoid sharp discontinuities in the boundary. The comparison of extensions of the input sequence between DCT and DFT is illustrated in Fig. 4.2.

4.2.2 Properties of DCT

DCT has several important properties, including separability, symmetry, orthogonality, and recursion.

The separation property of DCT is a valuable feature that allows the transform of a two-dimensional signal to be computed as a series of one-dimensional transforms.

Specifically, the two-dimensional DCT of an $N \times N$ signal $f(x, y)$ can be computed as follows:

$$\begin{aligned} F(u, v) &= \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \cos \left[\frac{(2y+1)v\pi}{2N} \right] \\ &= \sum_{x=0}^{N-1} C_u(x) \sum_{y=0}^{N-1} C_v(y) f(x, y), \end{aligned} \quad (4.5)$$

where $C_u(x) = \cos \left[\frac{(2x+1)u\pi}{2N} \right]$ and $C_v(y) = \cos \left[\frac{(2y+1)v\pi}{2N} \right]$. This property can significantly reduce the computational complexity of the transform, making it more efficient to compute for large data sets. The separability property of DCT plays an important role in image and video compression, where two-dimensional signals need to be transformed efficiently.

Since DCT is an even function, its symmetry property in one period allows for the computation of the DCT to be simplified, as only half of the coefficients need to be computed. Specifically, the N -point DCT of a signal $f(n)$ can be defined as:

$$F(u) = \sum_{n=0}^{N-1} f(n) \cos \left[\frac{(2n+1)u\pi}{2N} \right] = 2 \sum_{n=0}^{N/2-1} f(n) \cos \left[\frac{(2n+1)u\pi}{2N} \right], \quad (4.6)$$

where $u = 0, 1, \dots, N-1$.

Furthermore, DCT is an orthogonal transform, which means that the transformed coefficients are uncorrelated with each other. Specifically, the N -point DCT of a signal $f(n)$ can be defined as:

$$F(u) = \sqrt{\frac{2}{N}} \cdot C(u) \sum_{n=0}^{N-1} f(n) \cos \left[\frac{(2n+1)u\pi}{2N} \right], \quad (4.7)$$

where

$$C(u) = \begin{cases} \frac{1}{\sqrt{2}}, & \text{if } u = 0, \\ 1, & \text{otherwise.} \end{cases} \quad (4.8)$$

Orthogonality is an important property for compression applications, as it allows for efficient coding of the transform coefficients using techniques such as run-length encoding or Huffman coding.

In addition to its separability, symmetry, and orthogonality properties, DCT is also recursive. The fast DCT algorithm is a recursive approach that applies DCT to smaller and smaller sub-blocks of data. This recursive approach reduces

the computational complexity of DCT from $O(N^2)$ to $O(N \log N)$, making it a powerful tool for data compression and signal processing applications.

In conclusion, DCT is a valuable transform with several important properties, including separability, symmetry, orthogonality, and recursion. These properties make DCT a powerful tool for signal processing and data compression applications, particularly in the domain of compression.

4.2.3 Application in Point Cloud Compression

DCTs are widely used in data compression, and we introduce two methods adopting DCTs for point cloud attribute compression in this subsection. Generally, DCTs can be used to transform the input data into frequency representation. The energy is more compacted into the low-frequency components, resulting in better compression performance.

In the AVS PCC standard, DCTs are adopted to improve the point cloud attribute compression [5], and the overview of the compression framework is shown in Fig. 4.3. Firstly, points of the point cloud are grouped based on their geometry information, i.e., points with the same higher bits of the Hilbert code are assigned into a group. Then, the attribute residual of each point is calculated using prediction methods. Next, a 1-D DCT is utilized to transform attribute residuals in each group

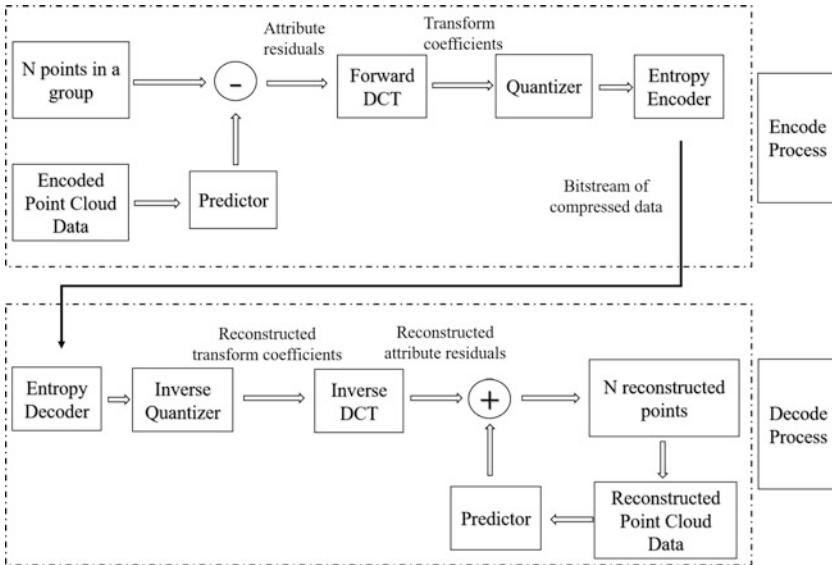


Fig. 4.3 The scheme of point cloud attribute compression in AVS PCC standard. (Source: Author)

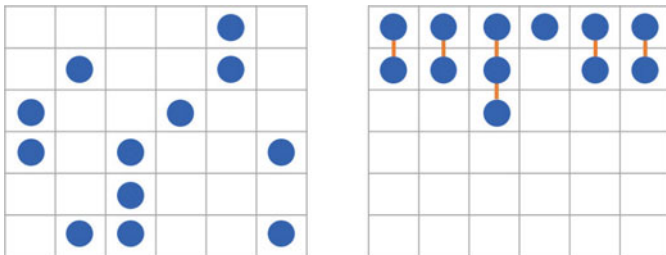


Fig. 4.4 The process of the shape-adaptive DCT in 2D. (Source: Author)

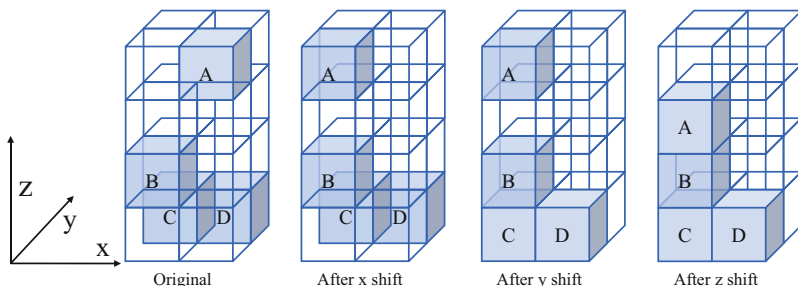


Fig. 4.5 An example of the shape-adaptive DCT on 3D point cloud. (Source: Author)

into coefficients in the frequency domain. For example, we have a group of attribute residuals as $R = [10, 13, 11]$. After applying a 3-ary DCT, the transform coefficient set is $C = [19.66, -0.71, -2.04]$. Notice that the information is compacted into the first few elements for each transform coefficient set. Finally, the transform coefficients are quantized and entropy-coded.

Another well-known work [6] proposed the shape-adaptive DCT for coding point cloud attributes. For the coding block with geometry and attribute information, they shift the points within this block line by line, as shown in the left of Fig. 4.4. It aims to achieve no empty positions in the block along the border, resulting in the right of Fig. 4.4. Then, the attribute value or prediction residuals of points along each occupied column are transformed with 1D DCTs. It starts at the block boundary and ends at the last data point where each column appears in the block. Filler values (e.g., 0) are inserted if it meets empty positions. Then, the shift and transform process are repeated along the remaining dimensions. Finally, each block obtained one DC and one or more AC coefficients. An example of a 3D point cloud is demonstrated in Fig. 4.5. Specifically, input signals $A, B, C, D = [3.2, 4.6, 3.8, 3.6]$ is shifted and transformed along the x direction, resulting in $A, B, C, D = [3.2, 4.6, 5.2326, 0.1414]$. Then, the shift and transform operations are repeated along y direction, its result is $A, B, C, D = [3.2, 4.6, 5.2326, 0.1414]$. Finally, the result after z shift is obtained as $A, B, C, D = [7.5244, -1.4373, -0.3133, 0.1414]$. At the decoder, the inverse shift and transform are done under the guidance of reconstructed geometry.

4.3 Wavelet Transform

Wavelet transform is a useful tool for signal processing that provides us with a high degree of freedom and flexibility. In this section, we begin with an introduction to the Fourier transform and then introduce the concept of wavelets. In comparison with the Fourier transform, wavelets are localized in both the time domain and the frequency domain, whereas the standard Fourier transform is only localized in frequency. So the wavelet transform provides multi-resolution representations, and one can analyze the different frequency components for the different time scales. Finally, the lifting transform is described, and a wavelet-based compression scheme of point cloud attribute compression is presented.

4.3.1 Fourier Transform

Fourier transform (FT) is a mathematical tool that analyzes a signal in the time domain by decomposing it into its frequency components [7]. It is commonly used in signal processing and other fields to analyze the frequency content of a signal. The Fourier transform is formally given as:

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-i\omega t} dt. \quad (4.9)$$

That is, the Fourier transform of an integrable function $f(t)$ at frequency ω is given by $F(\omega)$, usually called Fourier coefficients. The Fourier coefficients of the transformed function represent the contribution of each sine and cosine function at each frequency and are utilized in signal analysis. An inverse Fourier transform transforms data from the frequency domain back into the time domain as:

$$f(t) = \int_{-\infty}^{\infty} F(\omega) e^{i\omega t} d\omega. \quad (4.10)$$

For Fourier analysis, the frequency resolution of the time domain signal can be extremely high, while its time resolution tends to be zero. So we can't localize the information in the time domain. To overcome these drawbacks, the short-term Fourier transform (STFT) [8] is introduced. We divide a time signal into multiple pieces with equal time length and then apply the Fourier transform separately on each shorter piece. Given a time signal $f(t)$, STFT is defined as:

$$F(\omega, s) = \int_{-\infty}^{\infty} f(t) g(t-s) e^{-i\omega t} dt, \quad (4.11)$$

where $g(s)$ is the window function that is nonzero for only a short period of time. Unfortunately, STFT is difficult to invert due to its excessive redundancy,

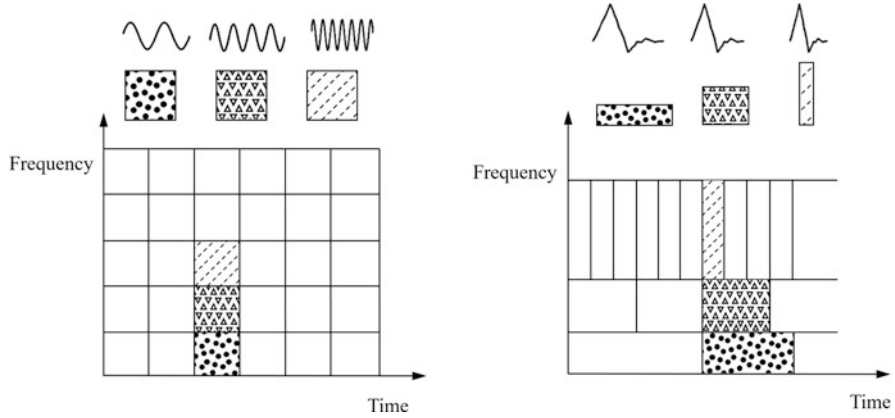


Fig. 4.6 Comparison between the short-term Fourier transform and wavelet transform. In STFT, the scale is fixed for all frequencies. In WT, the high-frequency signals are resolved at a low scale, whereas low-frequency signals are resolved at a high scale. ©1995 IEEE. Reprinted, with permission, from ref. [10]

and based on the Uncertainty Principle [9], it is unable to capture high-accuracy components simultaneously concentrated in both the time domain and frequency domain. Finally, a better solution is to use wavelets. The comparison between STFT and wavelet transform is shown in Fig. 4.6.

4.3.2 Wavelet Transform

Wavelets were mainly developed in the eighties and nineties. In wavelet analysis, we use a wavelet family, which is a collection of functions obtained by scaling and translating a wavelet, also called the Mother wavelet. The Mother wavelet $w(x)$ and its Fourier transform $\hat{w}(f)$ satisfy:

$$\int_{-\infty}^{\infty} w(x)dx = 0, \quad (4.12)$$

$$\int_{-\infty}^{\infty} \frac{|\hat{w}(f)|^2}{f} df = C_w < \infty. \quad (4.13)$$

The Eq. (4.13) is to ensure the invertibility of the wavelet transform. With Mother wavelet $w(x)$, a wavelet family is given as:

$$w_{a,b}(x) = \frac{1}{\sqrt{a}} w\left(\frac{x-b}{a}\right), \quad (4.14)$$

where a is positive and defines the scale, and b is any real number and defines the shift.

If a and b are continuous, then we have the continuous wavelet transform (CWT), which is defined as:

$$F(a, b) = \int_{-\infty}^{\infty} f(t) w_{a,b}(t) dt. \quad (4.15)$$

The inverse transform is given as:

$$f(t) = \frac{1}{C_w} \int_0^{\infty} \int_{-\infty}^{\infty} \frac{1}{a^2} F(a, b) w_{a,b}(t) db da, \quad (4.16)$$

where the constant C_w was defined in Eq. (4.13).

In practice, we always process the discrete signals leading to the discrete wavelet transform (DWT) [11]. For a Mother wavelet, one proper choice of the discrete version of a and b is as follows:

$$a = a_0^m, b = nb_0, \quad (4.17)$$

where $a_0 > 1$, $b_0 > 0$, m and n are integers. If let $b_0 = 2^m$ and $a_0 = 2$, we will obtain an orthogonal wavelet family. In this way, DWT is given by

$$F(m, n) = \int_{-\infty}^{\infty} f(t) w_{m,n}(t) dt, \quad (4.18)$$

$$w_{m,n}(t) = 2^{-m/2} w\left(\frac{t - n2^m}{2^m}\right). \quad (4.19)$$

The inverse DWT is defined as:

$$f(t) = \sum_{m,n} w_{m,n}(t) F(m, n). \quad (4.20)$$

We now introduce two popular wavelets with the proper choice of Mother wavelet. Haar wavelet is the simplest wavelet, developed by Alfréd Haar [12], and commonly used to analyze discrete signals. The Mother wavelet of Haar wavelet is defined as:

$$w(t) = \begin{cases} 1, & 0 \leq t < \frac{1}{2}, \\ -1, & \frac{1}{2} \leq t < 1, \\ 0, & \text{otherwise.} \end{cases} \quad (4.21)$$

When handling discrete signals, we use the Haar matrix. Generally, the $2N \times 2N$ Haar matrix can be computed recursively from 2×2 Haar matrix as:

$$H_{2N} = \begin{bmatrix} H_N \otimes [1, 1] \\ I_N \otimes [1, -1] \end{bmatrix}, H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad (4.22)$$

where I_N is $N \times N$ identity matrix, and $\otimes$ is the Kronecker product. Given an example, from Eq. (4.22), we have a 4×4 Haar transform matrix as:

$$H_4 = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ \sqrt{2} & -\sqrt{2} & 0 & 0 \\ 0 & 0 & \sqrt{2} & -\sqrt{2} \end{bmatrix}. \quad (4.23)$$

If we have an input signal $\mathbf{x} = [2, 5, 1, 3]^T$, the Haar transform coefficients can be calculated by:

$$\mathbf{y} = H_4 \mathbf{x} = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ \sqrt{2} & -\sqrt{2} & 0 & 0 \\ 0 & 0 & \sqrt{2} & -\sqrt{2} \end{bmatrix} \begin{bmatrix} 2 \\ 5 \\ 1 \\ 3 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 11 \\ 3 \\ -3\sqrt{2} \\ -2\sqrt{2} \end{bmatrix}. \quad (4.24)$$

The input signal $\mathbf{x}$ can be reconstructed by the inverse Haar transform

$$\hat{\mathbf{x}} = H_4^T \mathbf{y} = \frac{1}{4} \begin{bmatrix} 1 & 1 & \sqrt{2} & 0 \\ 1 & 1 & -\sqrt{2} & 0 \\ 1 & -1 & 0 & \sqrt{2} \\ 1 & -1 & 0 & -\sqrt{2} \end{bmatrix} \begin{bmatrix} 11 \\ 3 \\ -3\sqrt{2} \\ -2\sqrt{2} \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \\ 1 \\ 3 \end{bmatrix}. \quad (4.25)$$

Given a discrete signal with length 2^n , the Haar wavelet will divide the signal into multiple groups, then transform each group using the Haar matrix, hold the high-frequency components, and pass the low-frequency component. After repeating this process, final we obtain a final low-frequency component and $2^n - 1$ high-frequency components. The Haar wavelet family is shown in Fig. 4.7.

The Morlet wavelet is a popular wavelet composed of a complex exponential multiplied by a Gaussian [14]. The Mother wavelet is defined as:

$$w(t) = e^{-t^2/2} e^{ik_0 t}, \quad (4.26)$$

where k_0 is the predefined wavelet's central frequency. The Morlet wavelet can be considered a modulated Gaussian waveform if k_0 is large enough [15]. It is a complex wavelet that has a complex part and a purely real-valued part, as shown in Fig. 4.8.

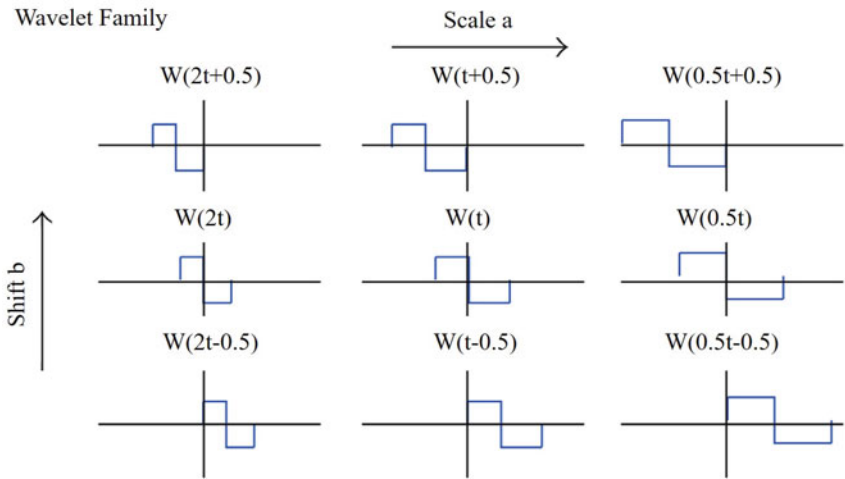


Fig. 4.7 Haar wavelet family. Public domain open access image [13]

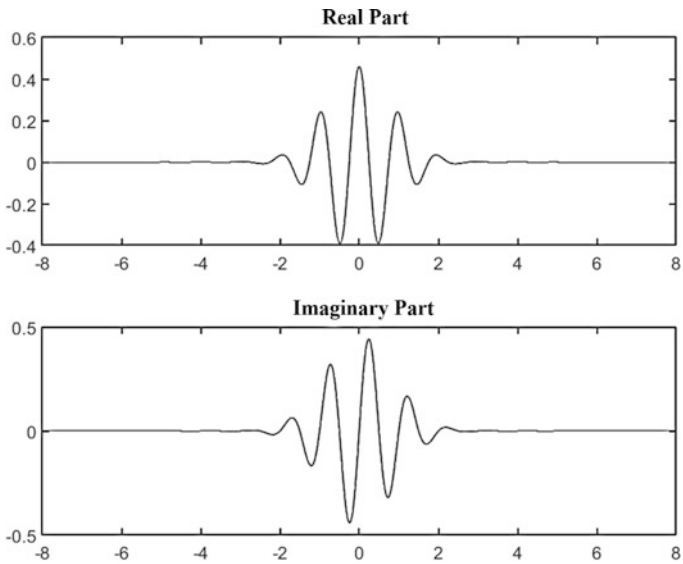


Fig. 4.8 The real and imaginary parts of the Morlet wavelet. (Source: Author)

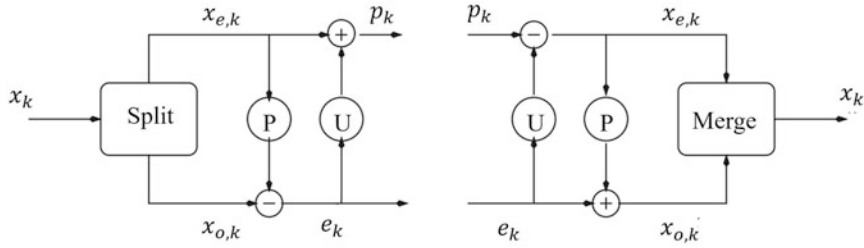


Fig. 4.9 Steps in the Lifting transform. “P” denotes the prediction step, and “U” denotes the update step. (Source: Author)

4.3.3 Lifting Transform

The lifting scheme was first introduced by Wim Sweldens as the second-generation wavelet transform [16]. The classical lifting scheme is composed of three steps, as shown in Fig. 4.9.

Split

Given a discrete signal x_k , $k \in \mathbb{Z}$, splits the signal into two. One common way is to separate a sequence into its even-indexed and odd-indexed components, and we denote the odd-sample signal by $x_{o,k}$ and the even-sample signal by $x_{e,k}$.

Prediction

The odd-sample signal can be predicted using the even samples. By subtracting the prediction from the odd-sample signal, an error signal e_k is obtained. If the signal is relatively slow changing, we tend to have accurate predictions and small error signals leading to a reduction of the data rate.

Update

The odd-sample signal $x_{o,k}$ is updated based on $x_{e,k}$ and e_k . We denote the updated signal as p_k . We can treat e_k as a high-frequency component of the original signal, while $x_{o,k}$ is simply the sampled version of the original signal. In order to reduce the aliasing caused by subsampling, the update operation is designed. One simple design is to preserve the average value of the signal.

It is an invertible scheme, as shown in the right part of Fig. 4.9. In the inverse process, the update step is applied first, with its result added back to the $x_{o,k}$, and then the same prediction is used to reconstruct the odd samples. Finally, the Merge

step is adopted as the opposite function of the split step to recover the original signal x_k .

Given an example [17], we can use the simplest prediction method, which estimates $x_{o,k}$ by using $x_{e,k}$. We have

$$e_k = x_{o,k} - x_{e,k}. \quad (4.27)$$

We can treat e_k as the high-frequency component of the original sequence. Then we can define a simple update operation as follows:

$$p_k = x_{e,k} + \lambda e_k. \quad (4.28)$$

We desire to preserve the average value of the original signal. Because the number of elements in the original sequence x_k is twice as many as that in the sequence p_k , we have

$$\sum_k x_k = 2 \sum_k p_k = 2 \sum_k (x_{e,k} + \lambda e_k), \quad (4.29)$$

where the summation range for the first sum is twice that of the other sum.

One possible solution is

$$\lambda = \frac{1}{2}, p_k = \frac{1}{2}(x_{e,k} + x_{o,k}). \quad (4.30)$$

This is just the implementation of Haar wavelets. We can treat p_k as the low-frequency component of the original sequence.

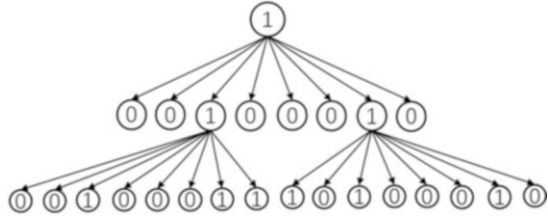
4.3.4 Application in Point Cloud Compression

Wavelets have many applications, and we examine the procedures of wavelet-based point cloud compression in this subsection. Region-adaptive hierarchical transform (RAHT) is one of the most popular wavelet-based methods for point cloud attribute compression [18]. It is a hierarchical sub-band transform that resembles an adaptive variation of a Haar wavelet.

In order to apply wavelet transform on signals over irregular domains of definition, the point cloud is firstly decomposed based on octree structure, as shown in Fig. 4.10. Then a variant Haar transform is applied in a bottom-up manner. At each level, instead of grouping eight voxels at a time, the transform is applied to two voxels in three steps along each dimension. In each step, the transform is given as:

$$\begin{bmatrix} y_l \\ y_h \end{bmatrix} = T_{w_1, w_2} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}, \quad (4.31)$$

Fig. 4.10 An octree decomposition of the point clouds. Occupied sub-nodes are further subdivided and denoted by “1.” Unoccupied sub-nodes are leaf nodes of the tree denoted by “0”. (Source: Author)



where a_1 and a_2 denote the attribute signal defined on two input voxels. The y_l are the low-pass coefficients and will be transformed in the next step. y_h are the high-pass coefficients and will be further quantized and entropy-encoded. T_{w_1, w_2} is calculated as:

$$T_{w_1, w_2} = \frac{1}{\sqrt{w_1 + w_2}} \begin{bmatrix} \sqrt{w_1} & \sqrt{w_2} \\ -\sqrt{w_2} & \sqrt{w_1} \end{bmatrix}, \quad (4.32)$$

where w_1 and w_2 are computed by the actual number of leaf voxels that a_1 and a_2 represent, respectively. The transform is applied recursively along each dimension and each level. The final transform coefficients of N occupied voxels are composed of one low-pass coefficient from the root node and $N - 1$ high-pass coefficients. It can be seen that the process reduces to the Haar transform when the weights are uniform. It is the case when the region is regular, e.g., fully occupied.

4.4 Graph Fourier Transform

Traditional digital signal processing (DSP) [19] assumes that signals are sampled over regular grids. Obviously, it has limitations considering irregular and complex data formats. Graph signal processing (GSP) has been proposed and developed as an extension of DSP to overcome the limitations in recent years [20–23]. This section mainly introduces the GFT which is one of the efficient transform tools in GSP. GFT can provide the compact spectral representations applied in signal compression. Then, different graph structures are explored to interpret the classical and well-known transforms, like KLT and DCT. Finally, specific compression algorithms based on GFT are demonstrated.

4.4.1 Introduction to GFT

Each point in point clouds is viewed as one vertex in a graph, as shown in Fig. 4.11. A graph is represented with three elements $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathbf{W}\}$, including a vertex set $\mathcal{V}$, an edge set $\mathcal{E}$ connecting vertices, and a weight matrix $\mathbf{W}$. Each entry $w_{i,j}$ in $\mathbf{W}$

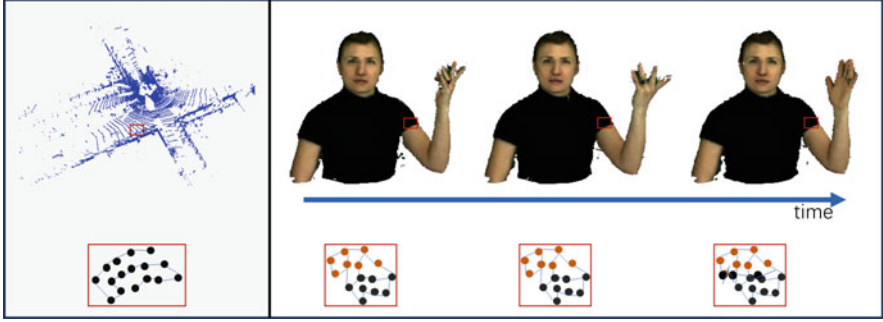


Fig. 4.11 Point cloud and its graph representation. The point clouds shown are introduced with MPEG open access (OA) work under CC BY Licence. (Copyright © 1988–2024, www.mpeg.org) [24]

represents the weight of the edge connecting vertices i and j , and it is explained as the similarity between adjacent vertices. Usually, each element in the weight matrix is non-negative. The combinatorial graph Laplacian L [25] is the common graph Laplacian matrix defined as:

$$L = D - W, \quad (4.33)$$

where D is the symmetric and diagonal degree matrix with elements $d_{i,i} = \sum_{j=1}^N w_{i,j}$ (N is the number of vertices in graph). Thus, L is a real symmetric matrix and supports the eigendecomposition, i.e.,

$$L = \mathbf{U} \Lambda \mathbf{U}^T, \quad (4.34)$$

where $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_N]$ is an orthonormal matrix consisting of eigenvectors $\mathbf{u}_1, \dots, \mathbf{u}_N$, and the corresponding eigenvalues $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_N)$ are viewed as the graph frequency spectrum.

Assuming that one signal $\mathbf{x} \in \mathbb{R}^N$ is defined as a vertex of the graph, its GFT is conducted as:

$$\hat{\mathbf{x}} = \mathbf{U}^T \mathbf{x}. \quad (4.35)$$

The inverse GFT is defined as:

$$\mathbf{x} = \mathbf{U} \hat{\mathbf{x}}. \quad (4.36)$$

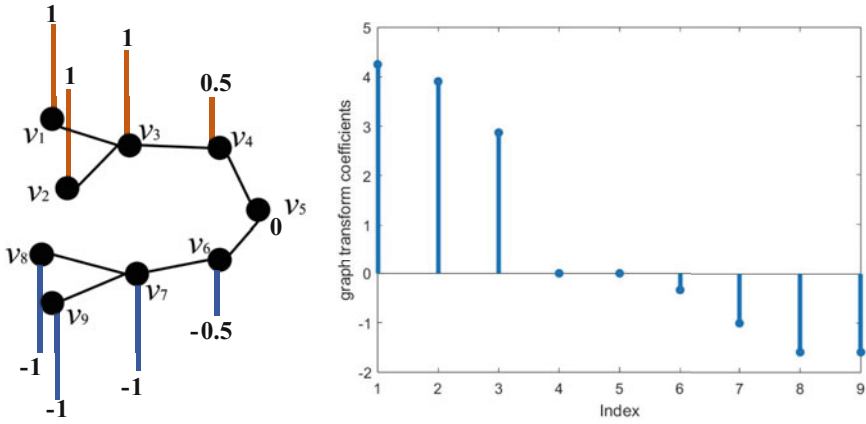


Fig. 4.12 The graph with edges weighted 1 and its graph Fourier transform coefficients. (Source: Author)

An example of GFT on signals is shown in Fig. 4.12, where the weight matrix W is represented as:

$$W = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}. \quad (4.37)$$

If the constructed graph weight can simulate the correlations between signals well, it is beneficial for GFT to obtain a compact representation and improve compression performance for graph signals.

Relation with KLT

The KLT is known to be the optimal transform that de-correlates signals with Gaussian entries. This subsection mainly demonstrates that KLT can be realized by GFT with respect to Gaussian Markov Random Field (GMRF).

A GMRF is a finite multivariate Gaussian distribution that satisfies additional conditional independence assumptions, and the assumption is generally represented with a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{W})$, where $\mathcal{V}$, $\mathcal{E}$ and $\mathbf{W}$ are node set, edge set, and symmetric non-negative weight matrix, respectively. It means that they are conditionally independent given all other nodes (i.e., $w_{i,j}$ and $w_{j,i}$ are set as 0) if nodes i and j are not directly connected. In contrast, the edge weight is assigned with a positive value when two nodes exist dependency.

Formally, color signals are modeled as a GMRF represented by graph $\mathcal{G} = (\mathbf{x} = \{x_1, \dots, x_N\}^T, \mathcal{E}, \mathbf{W})$ with probability density:

$$p(\mathbf{x}) = (2\pi)^{-\frac{N}{2}} |\mathbf{Q}|^{\frac{1}{2}} \exp \left(-\frac{1}{2} (\mathbf{X} - \mu)^T \mathbf{Q} (\mathbf{x} - \mu) \right), \quad (4.38)$$

and $Q_{ij} \neq 0 \Leftrightarrow \{i, j\} \in \mathcal{E}$ for all $i \neq j$,

where GMRF is characterized by the precision matrix $\mathbf{Q}$ and mean μ . In fact, precision matrix $\mathbf{Q}$ is the inverse of covariance matrix Σ , i.e., $\Sigma = \mathbf{Q}^{-1}$, and $\mathbf{Q}$ is utilized to interpret the conditional probability between signals. Furthermore, the optimal transform of decorrelating $\mathbf{x}$ is KLT with transform bases Ψ^T , where Ψ represents the eigenvectors of Σ . That can be defined as:

$$\Sigma \Psi = \Psi \Lambda, \quad (4.39)$$

where $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_N)$ is the diagonal matrix of eigenvalues of Σ . Since

$$\mathbf{Q} \Psi = \Sigma^{-1} \Psi = \left(\Psi \Lambda \Psi^T \right)^{-1} \Psi = \Psi \Lambda^{-1}, \quad (4.40)$$

the eigenvector matrix of $\mathbf{Q}$ is also Ψ . It indicates that $\mathbf{Q}$ can be treated as an alternative to covariance Σ while obtaining eigenvectors as the optimal transform bases. Moreover, $\mathbf{Q}$ is generally sparser than Σ due to the independent condition, which is beneficial to reduce the computational complexity [26]. Therefore, we derive $\mathbf{Q}$ to calculate its eigenvectors as transform bases of attributes. In graph signal processing, $\mathbf{Q}$ is equivalent to graph Laplacian $L = \mathbf{D} - \mathbf{W}$, where $\mathbf{D}$ is a diagonal matrix formulated as $d_{i,i} = \sum_{j=1}^N w_{i,j}$. Thus, the representation of the GMRF model is converted to define the appropriate weight matrix $\mathbf{W}$. The larger the weight value is, the stronger the correlation between nodes connected by this edge is.

Relation with DCT

Essentially, different types of DCTs can be interpreted by GFTs. The corresponding graphs are defined as some specific line structures. For example, DCT-2 can be

induced by the graph Laplacian matrix, and its weight matrix is represented in the following form:

$$\mathbf{W} = \begin{bmatrix} 0 & c & & 0 \\ c & 0 & c & \\ & \ddots & \ddots & \ddots \\ & & c & 0 & c \\ 0 & & & c & 0 \end{bmatrix}, \quad (4.41)$$

where $c > 0$. Readers can refer [27] to learn more about the mapping relation between different types of DCTs and graph Laplacian matrices. Obviously, certain DCT is constrained by one constant weight representation. It shows obvious limitations in representing the diverse correlation between natural signals. Therefore, it is necessary to construct an appropriate graph structure to flexibly and accurately simulate signal correlation.

4.4.2 Basics of Graph Spectral Compression

The Property of Transform

In a block-based compression framework, prediction and transform are two efficient and widely used technologies that reduce inter-block and intra-block redundancy, respectively. After the prediction and transform steps, quantization, and entropy coding are applied to code transform coefficients into the bitstream. In the encoder, transform technology is responsible for transferring signals from the space domain to the frequency domain. If transform coefficients are more concentrated on low frequency, more zeros are obtained after quantization, which is beneficial to improve the compression gains. Then, the key is how to design an efficient transform matrix. As shown above, the more the transform matrix can fit the statistical characteristics of the signal, the less the number of bases representing the whole signal after the transform. However, the statistical characteristic of the block is affected by the prediction mode and the video/point cloud content.

The Scheme of Graph Spectral Compression

Taking the point cloud attribute compression as an example, we exploit the reconstructed geometry information and auxiliary information as a reference and construct an identical graph structure at the encoder/decoder for GFT. (Assuming that the geometry and auxiliary information are coded separately via the specific coding algorithms.) The compression scheme applied in visual signal coding is demonstrated in Fig. 4.13. Graph construction is the key to the graph spectral

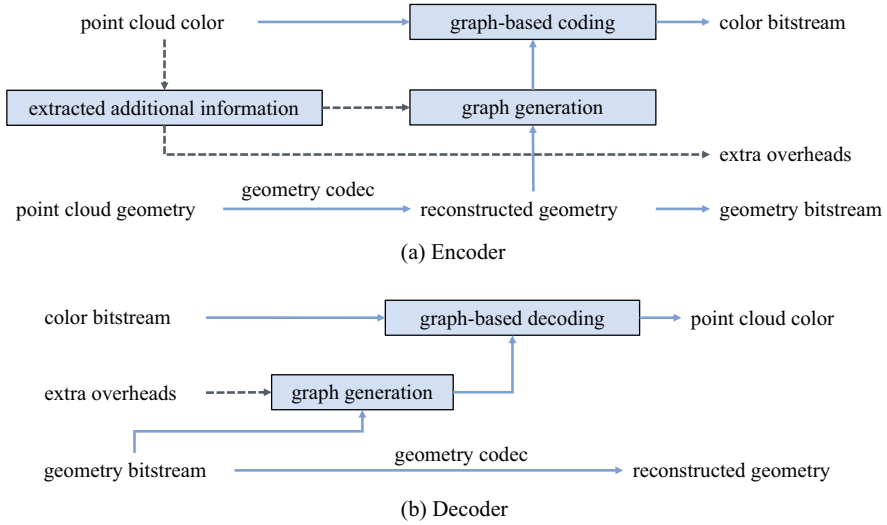


Fig. 4.13 General scheme of graph Fourier transform applied in point cloud compression. (Source: Author)

coding, which is generated with the geometry information or extra information obtained from color. It is important to note that this extra overhead needs to be written to the bitstream in order to reconstruct the graph structure at the decoder. Specifically, we demonstrate a process where a point cloud is encoded into bitstream in Fig. 4.14. First, a point cloud is partitioned into several blocks to reduce the computational complexity. Then, the reconstructed geometry of each block is used to construct a graph weight matrix W that models the attribute correlation as Eq. (4.37). Next, a graph Laplacian matrix L is calculated according to Eq. (4.33). The transform bases U are determined via the eigen decomposition on the Laplacian matrix L as depicted by Eq. (4.34). Then, the transform coefficients are obtained via the GFT of attribute values as Eq. (4.35). Finally, these transformed coefficients are quantized and entropy coded into the bitstream. Compared to attribute values in original space domain, the transform coefficients are more compact in the graph spectral domain. Since the majority of the energy is concentrated in the low frequency part of the spectrum domain as shown in Fig. 4.14, most transform coefficients in the high frequency part are turned to zero after quantization. As a result, GFT is an efficient tool for point cloud compression [28].

4.4.3 Application in Point Cloud Compression

As in the above subsection, graph construction plays a crucial role in GFT. The key to efficient compression is to generate a smooth graph for signals to obtain a more

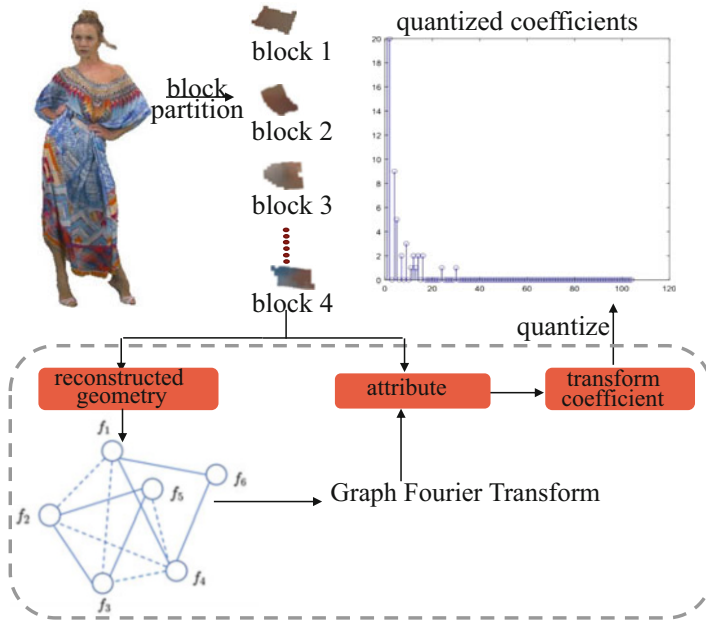


Fig. 4.14 An example of graph Fourier transform on a point cloud. The point cloud shown is introduced with MPEG open access (OA) work under CC BY Licence. (Copyright © 1988–2024, www.mpeg.org) [29]

compact representation in the spectral domain. Next, we introduce three methods that construct graphs for GFT in application to visual signal compression, including geometry-driven, feature-driven, and rate cost-driven graphs.

Geometry-Driven Graph

In some applications, the similarity between signals is related to the geometry distance between signals. Specifically, farther vertices would be less relevant, and closer vertices usually present a high similarity. To penalize signal values at a far distance, Gaussian function [30], inverse distance function [31], and other functions related to geometry distance are introduced to smooth the signal variations on the graph. As an example, point distribution is demonstrated in Fig. 4.15a. Its weight matrix is defined according to the inverse distance function as shown in Fig. 4.15b. The predefined threshold is set as 1.

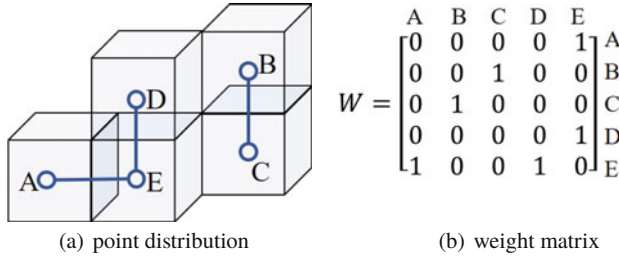


Fig. 4.15 Point distribution and corresponding weight matrix. (Source: Author)

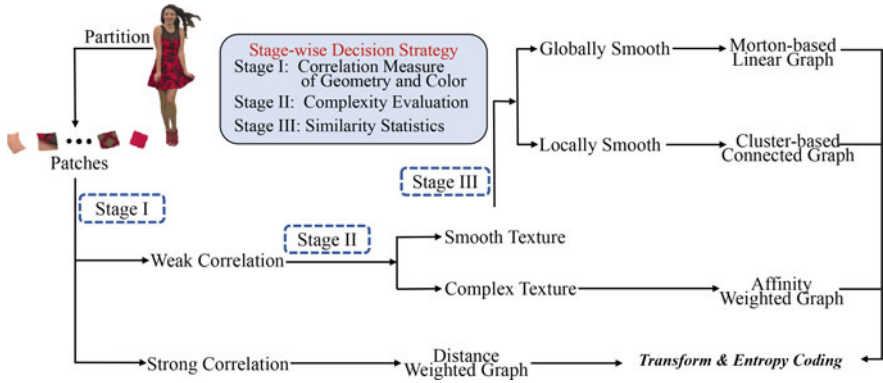


Fig. 4.16 Feature-based graph coding scheme. ©2022 IEEE. Reprinted, with permission, from ref. [32]

Feature-Driven Graph

The feature-driven graph is developed due to the following aspects: 1. The main hypothesis on the geometry-driven graph is that nodes are similar if they are close. However, this hypothesis is not always true when points cross the object boundary. Hence, it needs some auxiliary information to refine the graph structure. 2. The construction of the graph structure is not driven or determined by the input signals since it is not available at the decoder. If the input signals are directly used to guide the graph construction, input signals need to be encoded into the bitstream. Thus, there is an amount of bit cost. Therefore, the signal feature is represented with a few bits and utilized to refine the graph construction by analyzing the input signals. For example, Song et al. propose the fine-grained correlation graph representation [32]. The coding scheme is demonstrated in Fig. 4.16. It inspires that existing attribute compression methods based on GFT adopt the distance metric to define the Laplacian matrix. However, it does not represent the color correlation well in case of a poor relationship between geometry and color. Hence, considering point cloud color variation in space, it proposes three additional kinds of fine-grained correlation representation as Laplacian matrices for patches with different texture

categories. Furthermore, it utilizes texture complexity features prior to designing a stage-wise decision strategy for guiding each patch to choose appropriate correlation representation in low computation complexity. Experimental results demonstrate that this method achieves better compression performance compared with other platforms.

Rate Cost-Driven Graph

The above feature-driven graph introduces the auxiliary information based on the texture feature analysis. Although this method can improve the smoothness of the graph, it brings an extra bit of overhead. In order to balance the bit overheads and graph smoothness, many works focus on the rate cost optimization consisting of rate and distortion cost terms. The rate cost-driven graph-based codec [34] is classified as the rate cost-driven graph. Song et al. optimize the whole rate-distortion cost to determine the optimal adjacent matrix [33]. The optimization process is shown in Fig. 4.17, and the split-and-merge strategy is utilized to solve the convex problem based on RD cost. Another work proposes region-aware signal modeling to determine the optimal weight matrix for point cloud colors by minimizing the overall RD cost [35]. Its coding scheme is demonstrated in Fig. 4.18. Specifically, it considers the intuitive assumption that attribute variation is linearly correlated to the geometry variation in a local region. Thus, the reconstructed geometry can be used to analyze color correlation instead of the original colors. Thus, the coder can avoid coding the weight matrix. Moreover, the weight is optimal since it is obtained by solving the convex optimization problem. It is no longer limited to the massive pre-trained transform bases or inaccurate first-order approximation.

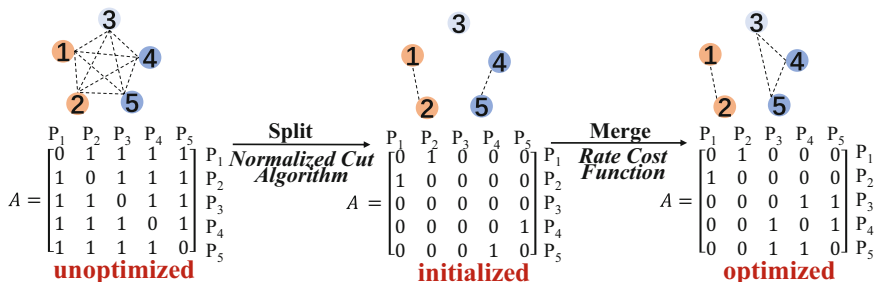


Fig. 4.17 Rate cost-based graph coding scheme via learning an adjacent matrix. ©2022 IEEE. Reprinted, with permission, from ref. [33]

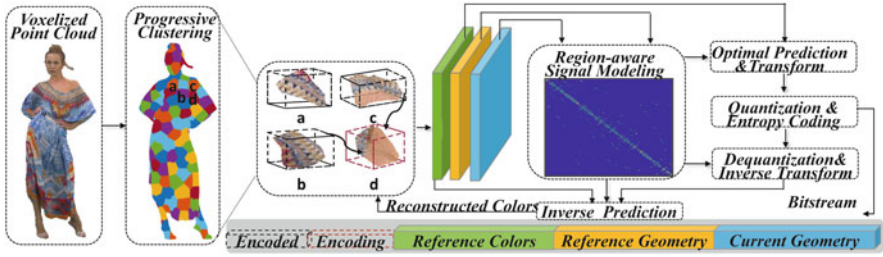


Fig. 4.18 Rate cost-based graph coding scheme via learning a weight matrix. ©2023 IEEE. Reprinted, with permission, from ref. [35]

4.5 Summary

In this chapter, we have described the concepts of Discrete Cosine Transform, Wavelets, and Graph Fourier Transform. We have shown how those transforms are applied to point cloud compression to provide an efficient decomposition of original signals. The transform-based compression approaches are currently areas of intensive research. Most works process the regular organized data, such as audio and image signals. However, point clouds are irregular and scattered in 3D space. Thus, it increases the difficulty of the compression task. Therefore, it is worthwhile to extend the traditional transform-based compression schemes on point clouds. Although graph spectral coding is one of the most popular and promising tools for point cloud compression, some problems still need to be solved. For example, spectral graph processing is highly complex due to storage and eigendecomposition. Specifically, GFT-based codecs all require the eigendecomposition of the Laplacian matrix. This operation brings heavy computational burdens and is unrealized in the case of large dimensions. Another important problem is the large-scale graph transform, e.g., how to reduce the graph transform size without affecting the performance. In the future, the light-weight design and accelerated implementations for transform coding [36, 37] can be effectively developed.

Exercises

1. What is the definition of Graph Fourier Transform?
2. Why not just find the covariance matrix of the sample to determine the optimal transformation instead of using the graph Laplacian matrix?
3. What is the key to designing codec based on graph transform?
4. Please describe the relation between the precision matrix and the graph Laplacian matrix.
5. Please discuss the complexity of the graph Fourier transform.
6. Please briefly describe the wavelet transform and its properties.

7. Please briefly describe the procedures and the advantages of the Lifting transform.
8. Given a signal

$$f(t) = 2 \cos(4t),$$

- (a) find and plot the Fourier transform $F(\omega)$ of $f(t)$
- (b) find and plot the STFT $F_s(\omega)$ of $f(t)$ using the window function

$$g(t) = \begin{cases} 1, & -4 \leq t \leq 4, \\ 0, & \text{otherwise.} \end{cases}$$

9. Please compute the un-normalized 8-point Haar matrix using Eq. (4.22).
10. Given a discrete signal sequence $f = \{2, 3, 1, 5, 4, 4, 3, 3\}$, apply the three-stage of the fast Haar-wavelet transform using the un-normalized 2-point Haar matrix to find the transform coefficient set.

References

1. J. Ballé, P.A. Chou, D. Minnen, S. Singh, N. Johnston, E. Agustsson, S.J. Hwang, G. Toderici, Nonlinear transform coding. *IEEE J. Sel. Top. Signal Proces.* **15**(2), 339–353 (2021)
2. H. Sun, T. Chiang, X. Chen, *Digital Video Transcoding for Transmission and Storage* (CRC Press, Boca Raton, 2004)
3. J.O. Smith, *Mathematics of the Discrete Fourier Transform (DFT): With Audio Applications* (Julius Smith, Stanford, 2008)
4. S.A. Khayam, *The Discrete Cosine Transform (DCT): Theory and Application* (Michigan State University, Michigan, 2003)
5. W. Zhang, J. Lu, N. Dai, T. Tian, Z. Lv, Core test report of point cloud attribute coding based on DCT transformation, Output document M6604, AVS, 78th AVS meeting, OnLine (2021)
6. R.A. Cohen, D. Tian, A. Vetro, Point cloud attribute compression using 3-D intra prediction and shape-adaptive transforms, in *Data Compression Conference* (2016), pp. 141–150
7. I.N. Sneddon, *Fourier Transforms* (Courier Corporation, North Chelmsford, 1995)
8. E. Sejdić, I. Djurović, J. Jiang, Time–frequency feature representation using energy concentration: an overview of recent advances. *Digital Signal Process.* **19**(1), 153–183 (2009)
9. D.L. Donoho, P.B. Stark, Uncertainty principles and signal recovery. *SIAM J. Appl. Math.* **49**(3), 906–931 (1989)
10. A. Graps, An introduction to wavelets. *IEEE Comput. Sci. Eng.* **2**(2), 50–61 (1995)
11. C.E. Heil, D.F. Walnut, Continuous and discrete wavelet transforms. *SIAM Rev.* **31**(4), 628–666 (1989)
12. A. Haar, *Zur Theorie der orthogonalen Funktionensysteme* (Georg-August-Universität, Göttingen, 1909)
13. A. Abou Nabout et al., Object shape recognition using wavelet descriptors. *J. Eng.* **2013**, 1–16 (2013)
14. J. Lin, L. Qu, Feature extraction based on morlet wavelet and its application for mechanical fault diagnosis. *J. Sound Vib.* **234**(1), 135–148 (2000)

15. D.T. Lee, A. Yamamoto, Wavelet analysis: theory and applications. Hewlett Packard J. **45**, 44–52 (1994)
16. W. Sweldens, Lifting scheme: a new philosophy in biorthogonal wavelet constructions, in *Wavelet Applications in Signal and Image Processing III*, vol. 2569 (1995), pp. 68–79
17. K. Sayood, *Introduction to Data Compression* (Morgan Kaufmann, Burlington, 2017)
18. R.L. De Queiroz, P.A. Chou, Compression of 3D point clouds using a region-adaptive hierarchical transform. *IEEE Trans. Image Process.* **25**(8), 3947–3956 (2016)
19. S.J. Orfanidis, *Introduction to Signal Processing* (Prentice-Hall, Inc., Hoboken, 1995)
20. A. Ortega, P. Frossard, J. Kovačević, J.M. Moura, P. Vanderghenst, Graph signal processing: overview, challenges, and applications. *Proc. IEEE* **106**(5), 808–828 (2018)
21. L. Stanković, M. Daković, E. Sejdić, Introduction to graph signal processing. *Vertex-Frequency Anal. Graph Signals*, 3–108 (2019)
22. A. Ortega, *Introduction to Graph Signal Processing* (Cambridge University Press, Cambridge, 2022)
23. W. Hu, J. Pang, X. Liu, D. Tian, C.-W. Lin, A. Vetro, Graph signal processing for geometric data and beyond: theory and applications. *IEEE Trans. Multimedia* **24**, 3961–3977 (2021)
24. C. Loop, Q. Cai, S.O. Escolano, P.A. Chou, Microsoft voxelized upper bodies – A voxelized point cloud dataset, Input document M38673/M72012, ISO/IEC JTC1/SC29 Joint WG11/WG1 (MPEG/JPEG), 115th MPEG meeting, Geneva (2016)
25. D.I. Shuman, S.K. Narang, P. Frossard, A. Ortega, P. Vanderghenst, The emerging field of signal processing on graphs: extending high-dimensional data analysis to networks and other irregular domains. *IEEE Signal Process Mag.* **30**(3), 83–98 (2013)
26. C. Zhang, D. Florencio, P.A. Chou, Graph signal processing - a probabilistic framework. Technical report, Microsoft Research (2015)
27. G. Strang, The discrete cosine transform. *SIAM Rev.* **41**(1), 135–147 (1999)
28. J. Zhang, Y. Chen, Q. Liu, W. Gao, G. Li, Efficient Point Cloud Attribute Compression Framework using Attribute-guided Graph Fourier Transform, in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (2022), pp. 8426–8430
29. D. Eugene, H. Bob, M. Taos, A.C. Philip, 8i voxelized full bodies, version 2 – a voxelized point cloud dataset, Input document M40059, ISO/IEC JTC1/SC29/WG7, 117th MPEG meeting, Geneva (2017)
30. Y. Shao, Q. Zhang, G. Li, Z. Li, L. Li, Hybrid point cloud attribute compression using slice-based layered structure and block-based intra prediction, in *ACM International Conference on Multimedia* (2018), pp. 1199–1207
31. C. Zhang, D. Florêncio, Analyzing the optimality of predictive transform coding using graph-based models. *IEEE Signal Process. Lett.* **20**(1), 106–109 (2012)
32. F. Song, G. Li, X. Yang, W. Gao, T.H. Li, Fine-grained correlation representation for graph-based point cloud attribute compression, in *IEEE International Conference on Multimedia and Expo* (2022), pp. 1–6
33. F. Song, G. Li, W. Gao, T.H. Li, Rate-distortion optimized graph for point cloud attribute coding. *IEEE Signal Process. Lett.* **29**, 922–926 (2022)
34. M. Rizkallah, T. Maugey, C. Guillemot, Rate-distortion optimized graph coarsening and partitioning for light field coding. *IEEE Trans. Image Process.* **30**, 5518–5532 (2021)
35. F. Song, G. Li, X. Yang, W. Gao, S. Liu, Block-adaptive point cloud attribute coding with region-aware optimized transform. *IEEE Trans. Circuits Syst. Video Technol.* **33**(8), 4294–4308 (2023)
36. W. Gao, H. Yuan, Y. Guo, L. Tao, Z. Cai, G. Li, OpenHardwareVC: an open source library for 8K UHD video coding hardware implementation, in *ACM International Conference on Multimedia* (2022), pp. 7339–7342
37. Y. Guo, W. Gao, S. Ma, G. Li, Accelerating transform algorithm implementation for efficient intra coding of 8K UHD videos. *ACM Trans. Multimedia Comput. Commun. Appl.* **18**(4), 1–20 (2022)